Natural Language Processing

Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 6

Vector Semantics & Embeddings

Word Meaning

What do words mean?

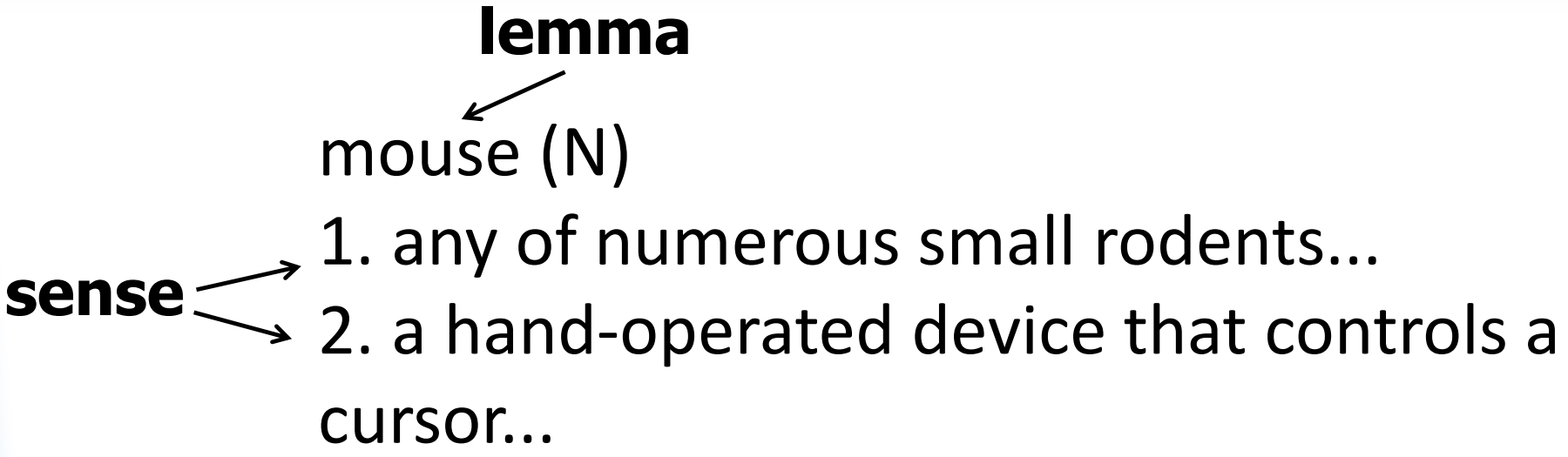
- N-gram or text classification methods we've seen so far
  - Words are just strings (or indices w_i in a vocabulary list)
  - That's not very satisfactory!
- Introductory logic classes:
  - The meaning of "dog" is DOG; cat is CAT

    $\forall x\ \mathrm{DOG}(x) \longrightarrow \mathrm{MAMMAL}(x)$
- Old linguistics joke by Barbara Partee in 1967:
  - Q: What's the meaning of life?
  - A: LIFE
- That seems hardly better!

Desiderata

- What should a theory of word meaning do for us?
- Let's look at some desiderata
- From **lexical semantics**, the linguistic study of word meaning

Lemmas and senses



- A **lemma** is the base or canonical form of a word, from which all its inflected forms are derived.
- A **sense** or "**concept**" is the meaning component of a word
- Lemmas can be **polysemous** (have multiple senses)

Relations between senses: Synonymy

- Synonyms have the same meaning in some or all contexts.
  - filbert / hazelnut
  - couch / sofa
  - big / large
  - automobile / car
  - vomit / throw up
  - water / H_20

Relations between senses: Synonymy

- There are probably no examples of perfect synonymy.
  - Even if many aspects of meaning are identical
  - Still may differ based on politeness, slang, genre, etc.

water/H_20

"H_20" in a surfing guide?

big/large

my big sister != my large sister

The Linguistic Principle of Contrast

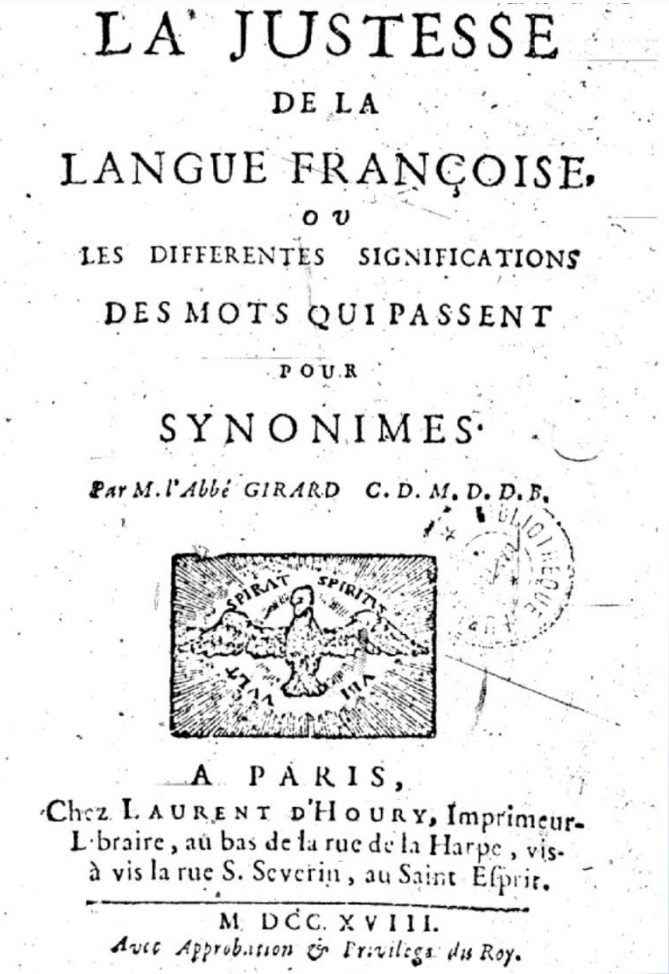
One of the fundamental tenets of semantics is called the **principle of contrast**:

Difference in form → difference in meaning

Abbé Gabriel Girard 1718

je ne crois pas qu'il y ait de-
"mot ſynonime dans aucune
Langue."

[I do not believe that there is a synonymous word in any language]


LA JUSTESSE
DE LA
LANGUE FRANÇOISE,
OU
LES DIFFERENTES SIGNIFICATIONS
DES MOTS QUI PASSENT
POUR
SYNONIMES.
Par M. l'Abbé GIRARD C. D. M. D. D. F.
SPIRAT SPIRITVS VITA VITA
A PARIS,
Chez LAURENT D'HOURY, Imprimeur-Libraire, au bas de la rue de la Harpe, vis-à-vis la rue S. Severin, au Saint Eſprit.
M. DCC. XVIII.
Avec Approbation & Privilege du Roy.

Thanks to Mark Aronoff!

Relation: Similarity

- Words with similar meanings. Not synonyms, but sharing some element of meaning

  car, bicycle

  cow, horse

- Knowing how similar two words are can help in computing how similar the meaning of two phrases or sentences are.

Ask humans how similar 2 words are

word1	word2	similarity
vanish	disappear	9.8
behave	obey	7.3
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

SimLex-999 dataset (Hill et al., 2015): Rating of the similarity relation on a scale from 0 to 10

Relation: Word relatedness

- Also called "word association“ in psychology
- Words can be related in any way, perhaps via a semantic frame or field

  - coffee, tea: **similar**
  - coffee, cup: **related**, not similar

Semantic field

- Words that
  - cover a particular semantic domain
  - bear structured relations with each other.

hospitals

surgeon, scalpel, nurse, anaesthetic, hospital

restaurants

waiter, menu, plate, food, menu, chef

houses

door, roof, kitchen, family, bed

Relation: Antonymy

- Senses that are opposites with respect to only one feature of meaning
- Otherwise, they are very similar!

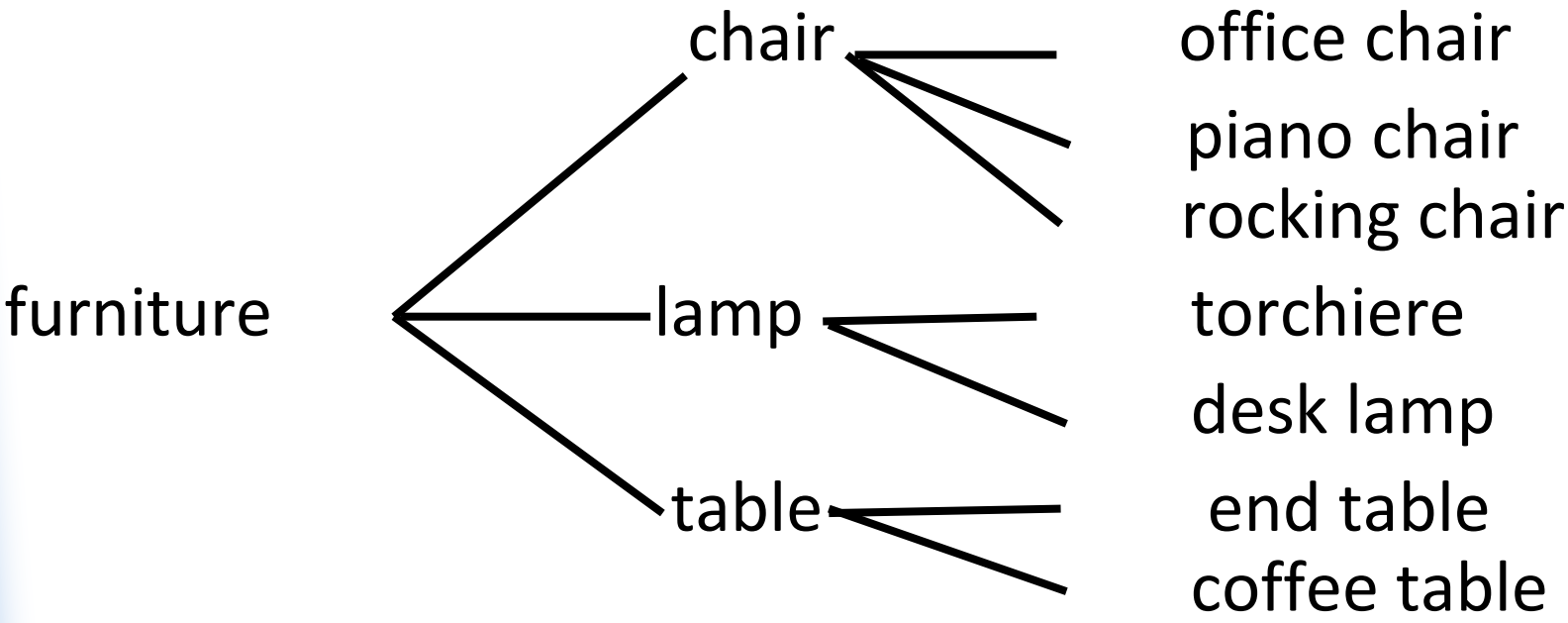
  dark/light short/long fast/slow rise/fall

  hot/cold up/down in/out
- More formally: antonyms can
  - define a binary opposition or be at opposite ends of a scale
    - long/short, fast/slow
  - Be *reversives*:
    - rise/fall, up/down

Relation: Superordinate/ subordinate

- One sense is a **subordinate** of another if the first sense is more specific, denoting a subclass of the other
  - *car* is a subordinate of *vehicle*
  - *mango* is a subordinate of *fruit*
- Conversely **superordinate**
  - *vehicle* is a superordinate of *car*
  - *fruit* is a superordinate of *mango*

Superordinate	vehicle	fruit	furniture
Subordinate	car	mango	chair


Superordinate
Basic
Subordinate
furniture
chair
lamp
table
office chair
piano chair
rocking chair
torchiere
desk lamp
end table
coffee table

Cluster of Interactional Properties

- Basic level things are “human-sized”
- Consider chairs
  - We know how to interact with a chair (sit)
  - Not so clear for superordinate categories like furniture
    - “Imagine a furniture without thinking of a bed/table/chair/specific basic-level category”

The basic level

- Distinctive actions
- Learned earliest in childhood
- Names are shortest
- Names are most frequent

Connotation (sentiment)

- Words have **affective** meanings
  - Positive connotations (*happy*)
  - Negative connotations (*sad*)
- Connotations can be subtle:
  - Positive connotation: *copy, replica, reproduction*
  - Negative connotation: *fake, knockoff, forgery*
- Evaluation (sentiment!)
  - Positive evaluation (*great*, *love*)
  - Negative evaluation (*terrible*, *hate*)

Connotation

Osgood et al. (1957)

- Words seem to vary along 3 affective dimensions:
  - **valence**: the pleasantness of the stimulus
  - **arousal**: the intensity of emotion provoked by the stimulus
  - **dominance**: the degree of control exerted by the stimulus

	Word	Score		Word	Score
Valence	love	1.000		toxic	0.008
	happy	1.000		nightmare	0.005
Arousal	elated	0.960		mellow	0.069
	frenzy	0.965		napping	0.046
Dominance	powerful	0.991		weak	0.045
	leadership	0.983		empty	0.081

Values from NRC VAD Lexicon (Mohammad 2018)

So far

- **Concepts** or word senses
  - Have a complex many-to-many association with **words** (homonymy, multiple senses)
- Have relations with each other
  - Synonymy
  - Antonymy
  - Similarity
  - Relatedness
  - Superordinate/subordinate, basic level
  - Connotation

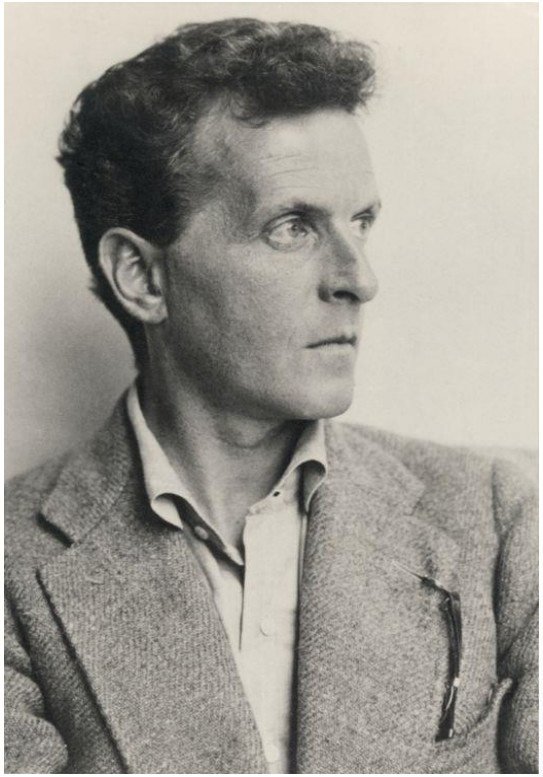
Vector Semantics

Computational models of word meaning

- Can we build a theory of how to represent word meaning, that accounts for at least some of the desiderata?
- We will introduce **vector semantics**
  - The standard model in language processing!
  - Handles many of our goals!

Ludwig Wittgenstein

- **"The meaning of a word is its use in the language"**
- He emphasized the idea that the meaning of words is not something intrinsic or fixed, but is determined by how they are used within specific social and linguistic contexts



Let's define words by their usages

- One way to define "usage": words are defined by their ***environments*** (the words around them)
- Zellig Harris (1954): **If A and B have almost identical environments we say that they are synonyms**.

Meaning of recent English borrowing *ongchoi*?

- Suppose you see these sentences:
  - Ong choi is delicious **sautéed with garlic**.
  - Ong choi is superb **over rice**
  - Ong choi **leaves** with salty sauces
- And you've also seen these:
  - …spinach **sautéed with garlic over rice**
  - Chard stems and **leaves** are **delicious**
  - Collard greens and other **salty** leafy greens
- Conclusion:
  - Ongchoi is a leafy green like spinach, chard, or collard greens
    - We could conclude this based on words like "leaves" and "delicious" and "sauteed"

Ongchoi: *Ipomoea aquatica "Water Spinach"*

空心菜
kangkong
rau muống
…



Yamaguchi, Wikimedia Commons, public domain

Idea 1: Defining meaning by linguistic distribution

- Let us define the meaning of a word by its distribution in language use, meaning its neighboring words or grammatical environments.

Idea 2: Meaning as a point in space

(Osgood et al. 1957)

- 3 affective dimensions for a word
  - **valence**: pleasantness
  - **arousal**: intensity of emotion
  - **dominance**: the degree of control exerted

	Word	Score		Word	Score
Valence	love	1.000		toxic	0.008
	happy	1.000		nightmare	0.005
Arousal	elated	0.960		mellow	0.069
	frenzy	0.965		napping	0.046
Dominance	powerful	0.991		weak	0.045
	leadership	0.983		empty	0.081

NRC VAD Lexicon (Mohammad 2018)

- Hence the connotation of a word is a vector in 3-space

Idea 1: Defining meaning by linguistic distribution

Idea 2: Meaning as a point in multidimensional space

Defining meaning as a point in space based on distribution

- Each word = a vector (not just "good" or "w_{45}")
- Similar words are "**nearby in semantic space**"
- We build this space automatically by seeing which words are **nearby in text** (unsupervised learning).



- The original 60-dimensional embeddings were trained for sentiment analysis.

Word meaning as a vector

- Called an "embedding" because it is embedded into a space (derived from its mathematical sense as a mapping from one space or structure to another)
- Standard way to represent meaning in NLP
- **Every modern NLP algorithm uses embeddings as the representation of word meaning**
- Fine-grained model of meaning for similarity

Intuition: why vectors?

- Consider sentiment analysis:
  - With **words**, a feature is a word identity
    - Feature 5: 'The previous word was "terrible"'
    - Requires **exact same word** to be in the training and test sets
  - With **embeddings**:
    - Feature is a word vector
    - 'The previous word was vector [35,22,17…]
    - Now in the test set we might see a similar vector [34,21,14]
    - We can generalize to **similar but unseen** words!!!

We'll discuss 2 kinds of embeddings

- **tf-idf**
  - Information Retrieval workhorse!
  - A common baseline model
  - **Sparse** vector representation
  - Words are represented by (a simple function of) the **counts** of nearby words
- **Word2vec**
  - **Dense** vectors
  - Representation is created by training a classifier to predict whether a word is likely to appear nearby
  - Later we'll discuss extensions called **contextual embeddings**

Computing with meaning representations

From now on: we will be computing with meaning representations instead of string representations

荃者所以在鱼，得鱼而忘荃 Nets are for fish;
Once you get the fish, you can forget the net.
言者所以在意，得意而忘言 Words are for meaning;
Once you get the meaning, you can forget the words

庄子(Zhuangzi), Chapter 26

Words and Vectors

Term-document matrix

- **Vector or distributional models of meaning** are generally based on a **co-occurrence matrix**, a way of representing how often words co-occur.
- In **term-document matrix**, each document is represented by a vector of words (columns below)

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

The term-document matrix for four words in four Shakespeare plays

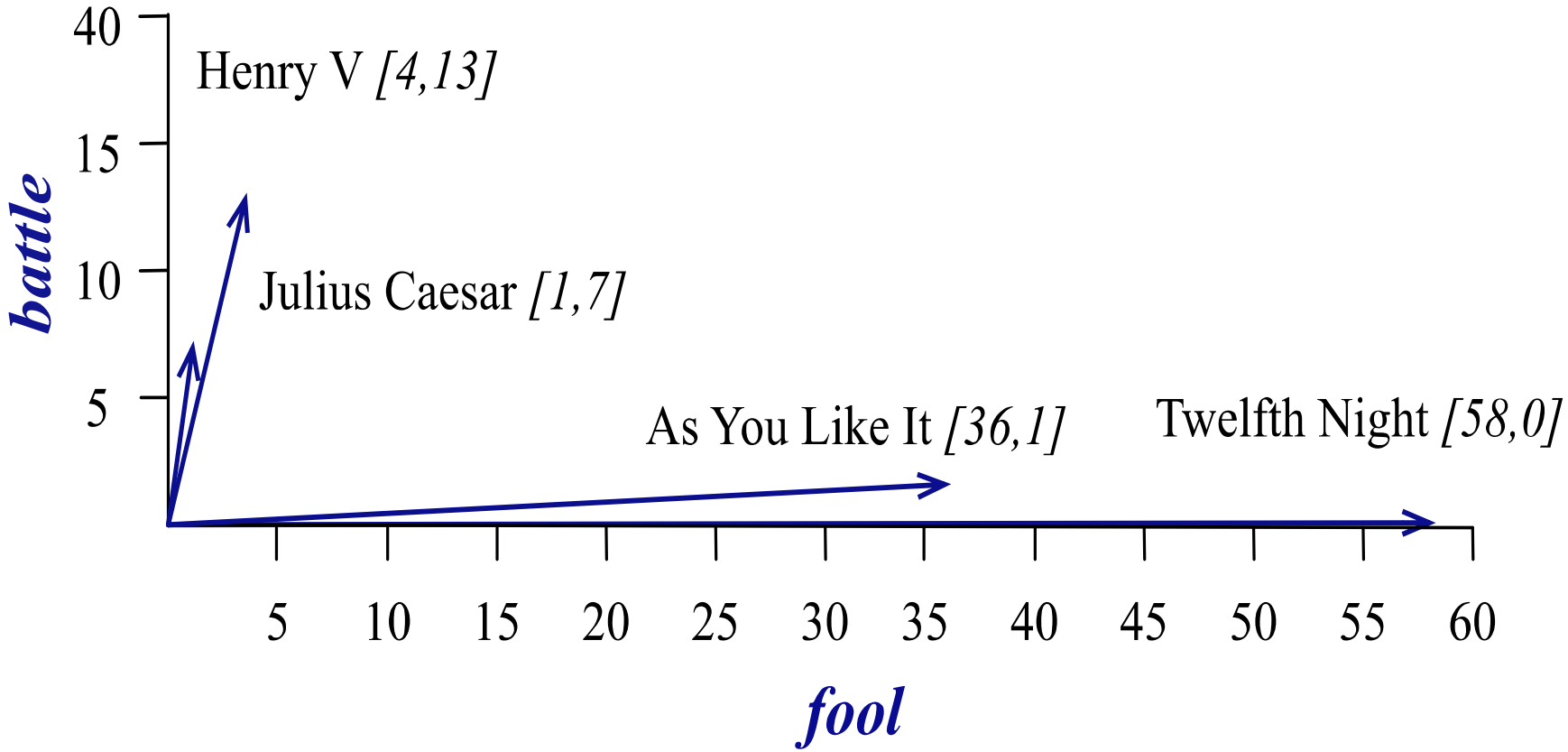
Vector Spaces

- Document vector representation in the example:
  - *As You Like It:* represented as [1,114,36,20]
  - *Julius Caesar*: represented as [7,62,1,2]
- A vector space is a collection of vectors, and is characterized by its dimension.
- Here document vectors are of dimension 4.
- In real term-document matrices, the document vectors would have dimensionality |V|, the vocabulary size.
- We can think of the vector for a document as a point in |V|-dimensional space.

Document Similarity

- **Term-document matrices** originally defined as a means of **finding similar documents** for the task of *document information retrieval*.
- Two documents that are similar will tend to have similar words, and
- If two documents have similar words their column vectors will tend to be similar.

Visualizing document vectors



Document vectors for the four Shakespeare play documents, showing just two of the dimensions

Vectors are the basis of information retrieval

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- Vectors are similar for the two comedies
- But comedies are different than the other two plays
  - Comedies have more *fools* and *wit* and fewer *battles*.

Idea for word meaning: Words can be vectors too!!!

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- *battle* is "the kind of word that occurs in Julius Caesar and Henry V"
- *fool* is "the kind of word that occurs in comedies, especially Twelfth Night"

More common: word-word matrix (or "term-context matrix")

- Two **words** are similar in meaning if their context vectors are similar.
- So have a co-occurrence matrix.

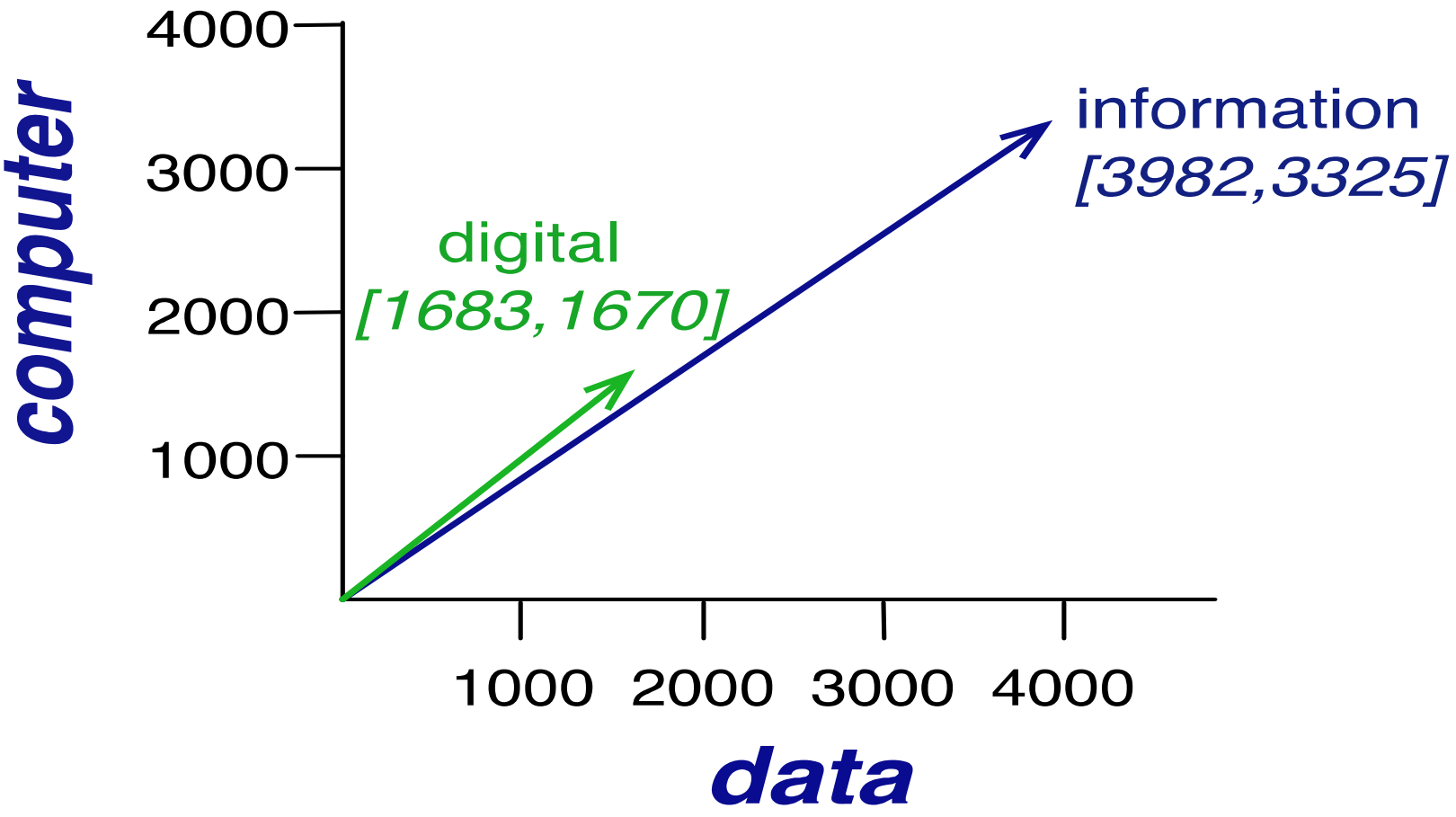
is traditionally followed by	**cherry**	pie, a traditional dessert
often mixed, such as	**strawberry**	rhubarb pie. Apple pie
computer peripherals and personal	**digital**	assistants. These devices usually
a computer. This includes	**information**	available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

word-word matrix (or "term-context matrix")

- This matrix is of dimensionality |V|x|V|
- Each cell records the number of times the row (target) word and the column (context) word co-occur in some context in some training corpus.
- The context could be the document.
- Most common to use smaller contexts, generally a window around the word (e.g. 4 words to the left and 4 words to the right).

Spatial Visualisation of Word Vectors



Spatial visualization of word vectors for ***digital*** and ***information***, showing just 2 of the dimensions (corresponding to ***data*** and ***computer***)

Cosine for computing word similarity

Computing word similarity: Dot product and cosine

- The dot product between two vectors is a scalar:

$$\text{dot product}(\mathbf{v},\mathbf{w}) = \mathbf{v}\cdot\mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N$$

- The dot product tends to be high when the two vectors have large values in the same dimensions
- Dot product can thus be a useful similarity metric between vectors

Problem with raw dot-product

- Dot product favors long vectors
- Dot product is higher if a vector is longer (i.e. has higher values in any dimension)
- Vector length:

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^{N} v_i^2}$$

- Frequent words (*of*, *the*, *you*) have long vectors (since they occur many times with other words).
- So dot product overly favors frequent words

Alternative: cosine for computing word similarity

$$\text{cosine}(\vec{v},\vec{w}) = \frac{\vec{v}\cdot\vec{w}}{|\vec{v}||\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

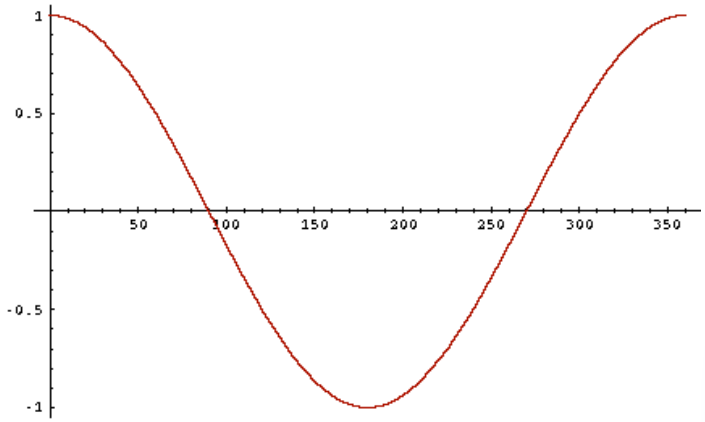
Based on the definition of the dot product between two vectors a and b

$$\mathbf{a}\cdot\mathbf{b} = |\mathbf{a}||\mathbf{b}|\cos\theta$$

$$\frac{\mathbf{a}\cdot\mathbf{b}}{|\mathbf{a}||\mathbf{b}|} = \cos\theta$$

Cosine as a similarity metric

- -1: vectors point in opposite directions
- +1: vectors point in same directions
- 0: vectors are orthogonal



- But since raw frequency values are non-negative, the cosine for term-term matrix vectors ranges from 0 to 1.

Cosine examples

$$\cos(\vec{v},\vec{w}) = \frac{\vec{v}\cdot\vec{w}}{|\vec{v}||\vec{w}|} = \frac{\vec{v}}{|\vec{v}|}\cdot\frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

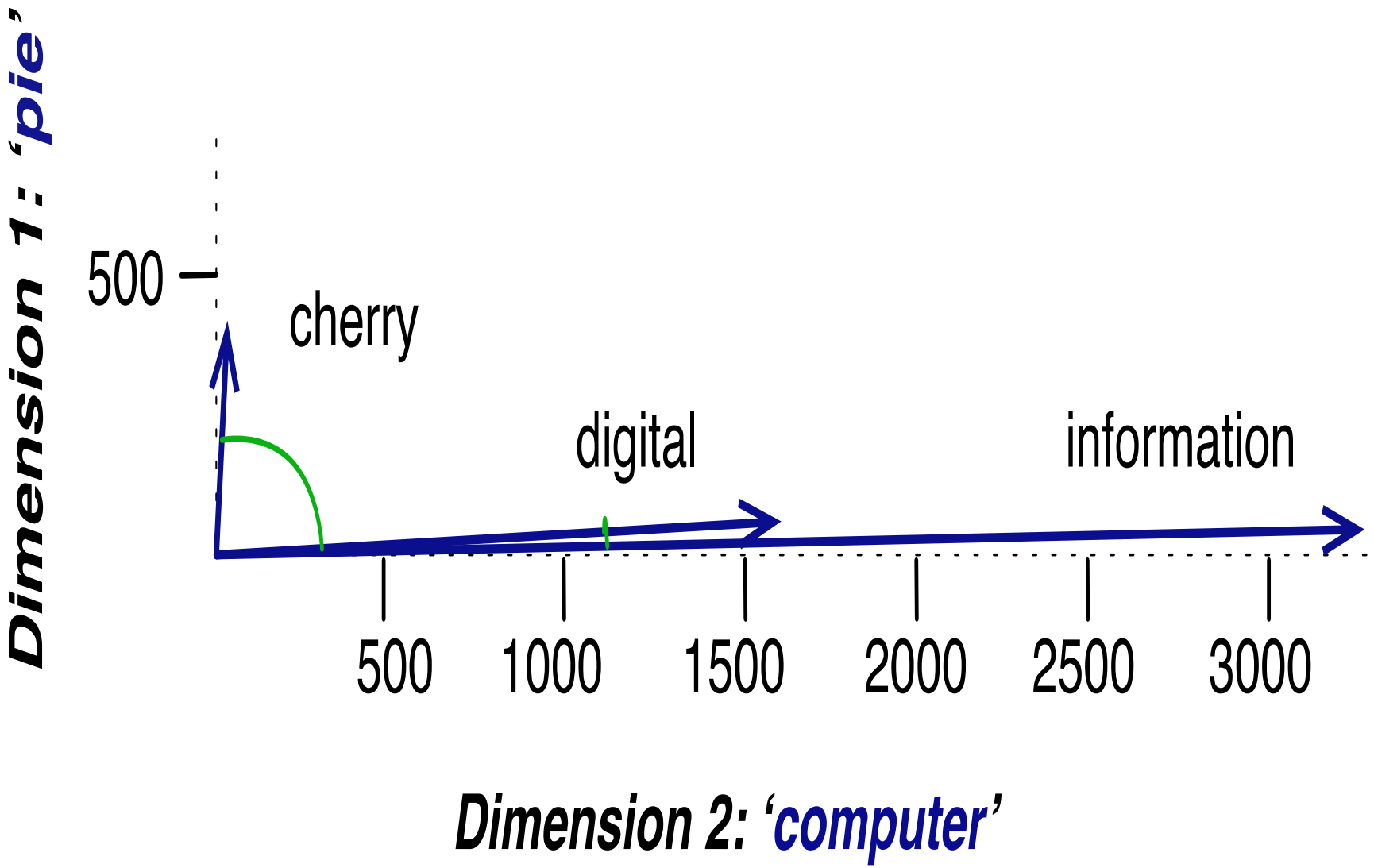
cos(cherry,information) =

$$\frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}} = .017$$

cos(digital,information) =

$$\frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}} = .996$$

Visualizing cosines (angles)



TF-IDF

Raw frequency is a bad representation

- The co-occurrence matrices we have seen represent each cell by word frequencies.
- Frequency is clearly useful; if *sugar* appears a lot near *apricot*, that's useful information.
- But overly frequent words like *the*, *it*, or *they* are not very informative about the context
- It's a paradox!
- How can we balance these two conflicting constraints?

Two common solutions for word weighting

- **tf-idf** (usually used when the dimensions are documents): the tf-idf value for word t in document d:

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

Words like "the" or "it" have very low idf

- **PMI** (Pointwise mutual information; usually used when the dimensions are words):

$$\text{PMI}(w_1, w_2) = log \frac{p(w_1,w_2)}{p(w_1)p(w_2)}$$

See if words like "good" appear more often with "great" than we would expect by chance

Term frequency (tf) in the tf-idf algorithm

- We could imagine using raw count:

  $\mathbf{tf}_{t,d} = \text{count}(t, d)$

- But instead of using raw count, we usually squash it a bit using the *log* function:

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t,d) & \text{if } \text{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Document frequency (df)

- $\mathbf{df}_t$ *is* the number of documents *t* occurs in.
- (Note that this is not collection frequency: total count across all documents)
- "*Romeo*" is very distinctive for one Shakespeare play:

	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

Inverse document frequency (idf)

$$\text{idf}_t = \log_{10}\left(\frac{N}{\text{df}_t}\right)$$

N is the total number of documents in the collection (N=37 here)

- Log is used because of the large number of documents in many collections.

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

What is a document?

- A document could be a play or a Wikipedia article
- For the purposes of tf-idf, documents can be **anything**; often each paragraph is called a document!

Final tf-idf weighted value for a word

- Raw counts:

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- tf-idf: $w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

Discussion of tf-idf example

- The idf weighting has eliminated the importance of the ubiquitous word *good*

  and

- The idf weighting has vastly reduced the impact of the almost-ubiquitous word *fool*.
- As a consequence, tf-idf(t,d) favors words (terms) that are distinctive of a document (d).

Summary on tf-idf and PPMI

- Vector semantics model so far:
  - a target word represented as a vector with dimensions corresponding to
    - the documents in a large collection (the term-document matrix) or to
    - the counts of words in some neighboring window (the term-term matrix).
  - The values in each dimension are counts, weighted by
    - tf-idf (for term-document matrices) or
    - PPMI (for term-term matrices), and the

Vector Semqntics

- Vectors are sparse (since most values are zero).
- The model computes the similarity between two words x and y by taking the cosine of their tf-idf or PPMI vectors;
- High cosine means high similarity

Document Vector

- The tf-idf model of meaning is often used for document functions, e.g. deciding if two documents are similar.
- A document is represented by taking the vectors of all the words in the document, and computing the **centroid** of all those vectors.
- Given k word vectors w_1, w_2, …, w_k, the centroid **document vector** *d* is:

$$d = \frac{w_1 + w_2 + \cdots + w_k}{k}$$

Document similarity and applications

- Given two documents, we can compute their document vectors d_1 and d_2, and estimate the similarity between the two documents by $\cos(d_1, d_2)$.
- Useful for various applications: information retrieval, plagiarism detection, news recommender systems, digital humanities tasks (e.g. comparing different versions of a text to see which are similar to each other).
- PPMI or tf-idf can be used for finding word paraphrases, tracking changes in word meaning, etc.
- Example; Finding 10 most similar words to some word?

Word2vec

Sparse versus dense vectors

- tf-idf (or PMI) vectors are
  - **long** (length |V|= 20,000 to 50,000)
  - **sparse** (most elements are zero)
- Alternatively: learn vectors which are
  - **short** (length 50-1000)
  - **dense** (most elements are non-zero)

Sparse versus dense vectors

- Why dense vectors?
  - Short vectors may be easier to use as **features** in machine learning (fewer weights to tune)
  - Dense vectors may **generalize** better
  - Dense vectors may do better at capturing synonymy:
    - In a sparse vector representation, dimensions for *car* and *automobile* (synonyms) are distinct and unrelated
      - a word with *car* as a neighbor and a word with *automobile* as a neighbor should be similar, but aren't
  - **In practice, dense vectors work better**

Common methods for getting short dense vectors

- "Neural Language Model"-inspired models
  - Word2vec (Skip-gram, CBOW), GloVe
- Singular Value Decomposition (SVD)
- These are Static embeddings approaches: i.e. they learn one fixed embedding for any vocabulary word.
- Alternative to these "static embeddings": Contextual (or dynamic) Embeddings (ELMo, BERT)
  - Compute distinct embeddings for a word sense based on the context

Simple static embeddings you can download!

- Word2vec (Mikolov et al)

https://code.google.com/archive/p/word2vec/

- GloVe (Pennington, Socher, Manning)

http://nlp.stanford.edu/projects/glove/

Word2vec

- Popular embedding method
- Very fast to train
- Code available on the web
- Idea: **predict** rather than **count**
- Word2vec provides various options. We'll do:

  Skip-gram with negative sampling (SGNS)

Word2vec

- Instead of **counting** how often each word *w* occurs near "*apricot*"
  - Train a classifier on a binary **prediction** task:
    - Is *w* likely to show up near "*apricot*"?
- We don't actually care about this task
    - But we'll take the learned classifier weights as the word embeddings
- The big idea is **self-supervision**:
    - A word c that occurs near apricot in the corpus acts as the gold "correct answer" for supervised learning
    - No need for human labels

  (Bengio et al. (2003); Collobert et al. (2011))

Approach: predict if candidate word *c* is a "neighbor"

1. Treat the target word *t* and a neighboring context word *c* as **positive examples**.
2. Randomly sample other words in the lexicon to get negative examples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as the embeddings

Skip-Gram Training Data

- Assume a +/- 2 word window, given the training sentence:

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 [target] c3 c4

Skip-Gram Classifier

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 [target] c3 c4

- **Goal**: train a classifier such that, given a candidate (**w**ord, **c**ontext) pair, e.g.

(apricot, jam)

(apricot, aardvark)

…

assigns each pair a probability: $P(+|w, c)$

$P(-|w, c) = 1 - P(+|w, c)$

Dot product to compute Similarity

- Intuition of the Skip-gram model:
  - base this probability on embedding similarity; i.e.
  - a word is likely to occur near the target if its embedding vector is similar to the target embedding.
- Recall that: two vectors are similar if they have a high dot product
  - Cosine is just a normalized dot product
- So:
  - Similarity(w, c) ≈ w · c
- Note: We'll need to normalize to get a probability
  - (cosine isn't a probability either)

Turning dot products into probabilities

- Sim(w,c) ≈ w · *c*
- To turn this into a probability, the sigmoid function is used:

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$$

$$P(-|w,c) = 1 - P(+|w,c)$$

$$= \sigma(-c \cdot w) = \frac{1}{1+\exp(c \cdot w)}$$

How the Skip-Gram classifier computes $P(+|w, c)$

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$$

- This is for one context word; but we have lots of context words.
- We'll assume independence and just multiply them:

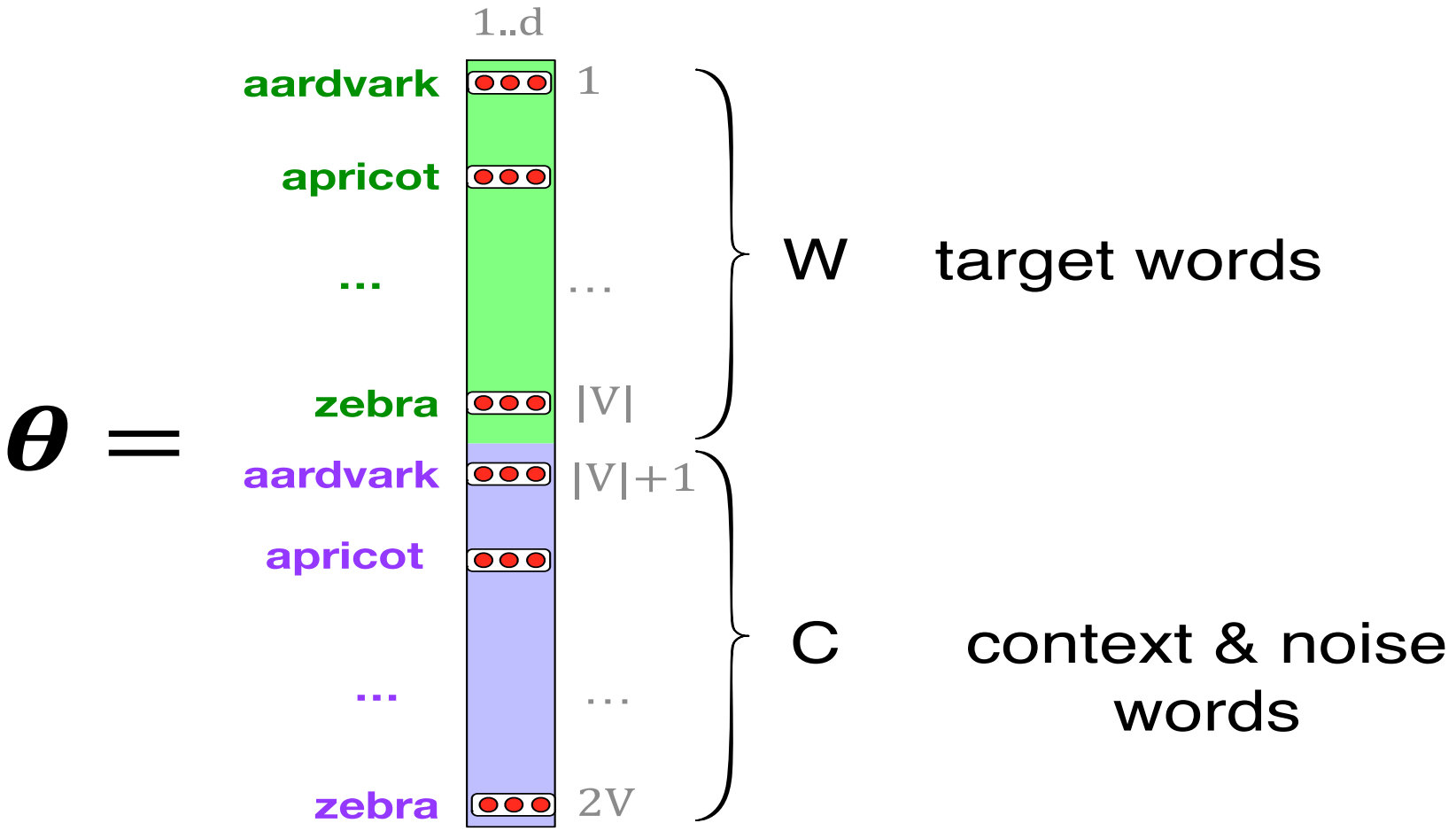
$$P(+|w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i \cdot w)$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^{L} \log \sigma(c_i \cdot w)$$

Skip-gram classifier: summary

- A probabilistic classifier, given
  - a test target word *w*
  - its context window of *L* words $c_{1:L}$
- It estimates the probability that w occurs in this window based on similarity of w (embeddings) to $c_{1:L}$ (embeddings).
- To compute this, we just need embeddings for all the words.

Needed embeddings: a set for w & a set for c



d is the dimension of each word embedding

Word2vec: Learning the Embeddings

Skip-Gram Training data

- Assuming a +/- 2 word window,

…lemon, a [tablespoon of apricot jam, a] pinch…
c1 c2 [target] c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

- In Skip-gram: for each positive example we'll take k negative examples, sampling by frequency.
- Suppose k = 2

Skip-Gram Training data

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 [target] c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

Word2vec: how to learn vectors

- Given the set of +ve and -ve training instances, and an initial set of embedding vectors
- The goal of learning is to adjust those word vectors such that we:
  - **Maximize** the similarity of the target word, context word pairs (w , c_{pos}) drawn from the positive data
  - **Minimize** the similarity of the (w , c_{neg}) pairs drawn from the negative data.

Loss function for one *w with*

c_{pos} , c_{neg1}, … , c_{negk}

- Maximize the similarity of the target with the actual context words, and minimize the similarity of the target with the *k* negative sampled non-neighbor words.
- The following loss function L is to be minimized

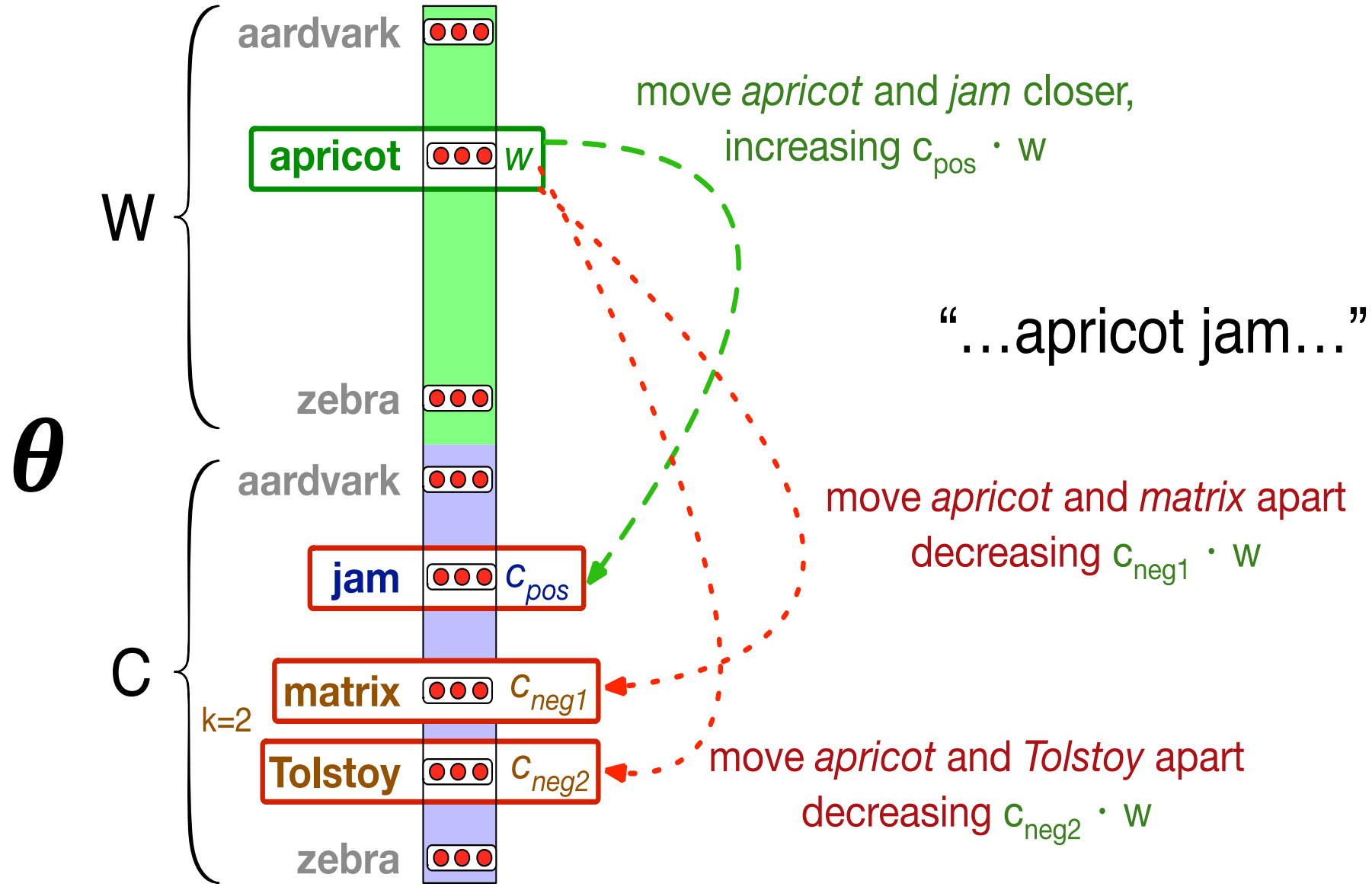
$$
\begin{aligned}
L_{CE} &= -\log\left[P(+|w,c_{pos})\prod_{i=1}^{k}P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log\left(1-P(+|w,c_{neg_i})\right)\right] \\
&= -\left[\log \sigma(c_{pos}\cdot w) + \sum_{i=1}^{k}\log\sigma(-c_{neg_i}\cdot w)\right]
\end{aligned}
$$

Learning the classifier

- How to learn it?
  - Use stochastic gradient descent!
- The word weights will be adjusted so as to make
  - the positive pairs more likely and
  - the negative pairs less likely,

  over the entire training set.

Intuition of one step of gradient descent



Reminder: gradient descent

- At each step
  - Direction: move in the reverse direction from the gradient of the loss function
  - Magnitude: move by the value of this gradient $\frac{d}{dw} L(f(x; w), y)$ weighted by a **learning rate** η (higher learning rate means move *w* faster)

$$w^{t+1} = w^t - h \frac{d}{dw} L(f(x; w), y)$$

The derivatives of the loss function

$$L_{CE} = -\left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^{k} \log \sigma(-c_{neg_i} \cdot w)\right]$$

To get the gradient, we need to take the derivative:

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^{k} [\sigma(c_{neg_i} \cdot w)]c_{neg_i}$$

Stochastic Gradient Descent

- **Initial State**: Initialize the parameters (weights) of the model. The matrices W and C represent these parameters in the context of Word2Vec.
  - **W**: The word vectors for each word when it's the **target word**.
  - **C**: The word vectors for each word when it's a **context word**.
- **Learning from Data**: As one goes through the training corpus, the Skip-gram model learns which word pairs (target, context) co-occur frequently and adjusts the embeddings in W and C to reflect the relationships between words.

Update equation in SGD

- The update equations going from time step t to t +1 in stochastic gradient descent are thus:

$$c_{pos}^{t+1} = c_{pos}^{t} - \eta\left[\sigma\left(c_{pos}^{t} \cdot w^{t}\right) - 1\right]w^{t}$$

$$c_{neg}^{t+1} = c_{neg}^{t} - \eta\left[\sigma\left(c_{neg}^{t} \cdot w^{t}\right)\right]w^{t}$$

$$w^{t+1} = w^{t} - \eta\left[\left[\sigma\left(c_{pos} \cdot w^{t}\right) - 1\right]c_{pos} + \sum_{i=1}^{k}\left[\sigma\left(c_{neg_i} \cdot w^{t}\right)\right]c_{neg_i}\right]$$

Two sets of embeddings

- Skip-gram with negative sampling (SGNS) learns two sets of embeddings
  - Target embeddings matrix W
  - Context embeddings matrix C
- It is common to just add them up, representing word *i* as the vector $w_i + c_i$
- Alternatively, one can throw away the C matrix and just represent each word i by the vector wi.
- The context window size **L** affects the performance of skip-gram embeddings; experiments often tune the parameter L on a devset.

Summary: How to learn word2vec (skip-gram) embeddings

- Start with V random d-dimensional vectors as initial embeddings
- Train a classifier based on embedding similarity
  - Take a corpus and take pairs of words that co-occur as positive examples
  - Take as negative examples pairs of words that don't co-occur
  - Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance
  - Throw away the classifier code and keep the embeddings.

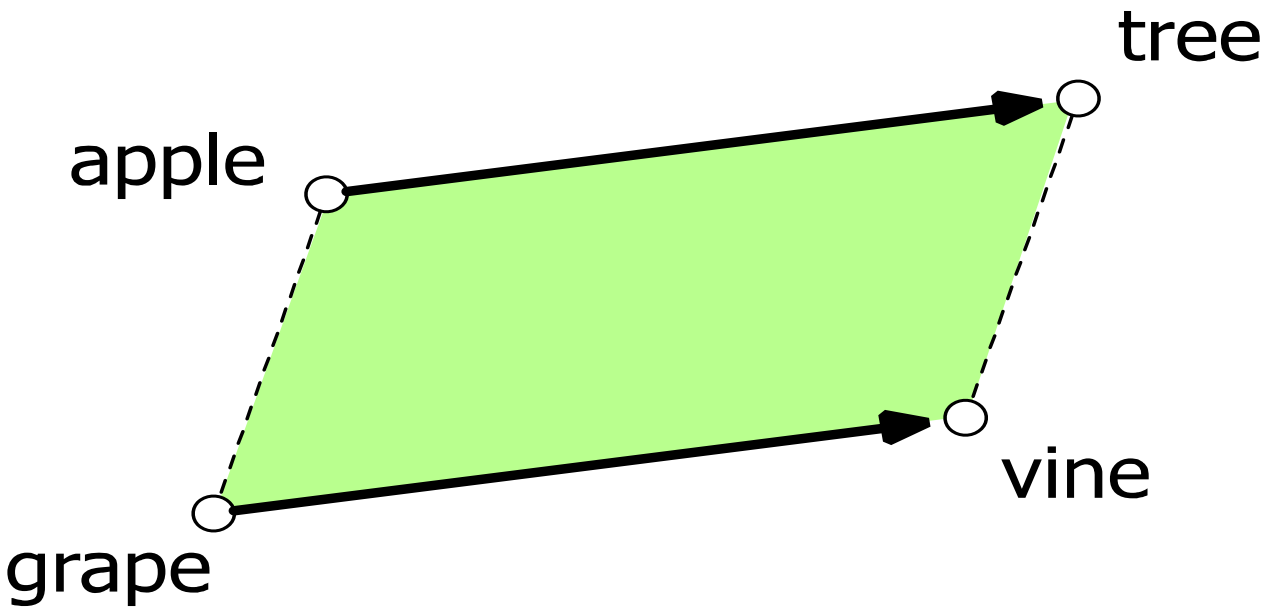
Properties of Embeddings

Window size and neighbors

- **Small windows** (C= +/- 2) : nearest words are syntactically similar words in the same taxonomy
  - *Hogwarts* nearest neighbors are other fictional schools
    - *Sunnydale, Evernight, Blandings*
- **Large windows** (C= +/- 5) : nearest words are related words in the same semantic field
  - *Hogwarts nearest neighbors are Harry Potter world:*
    - *Dumbledore, half-blood, Malfoy*

Analogical relations

- The classic parallelogram model of analogical reasoning (Rumelhart and Abrahamson 1973)
- To solve: *"apple is to tree as grape is to __?___"*
- *Add* $\overrightarrow{tree}$ – $\overrightarrow{apple}$ *to* $\overrightarrow{grape}$ *to get* ***vine*** $\overrightarrow{vine}$



Analogical relations via parallelogram

- The parallelogram method can solve analogies with both sparse and dense embeddings (Turney and Littman 2005; Mikolov et al. 2013b)

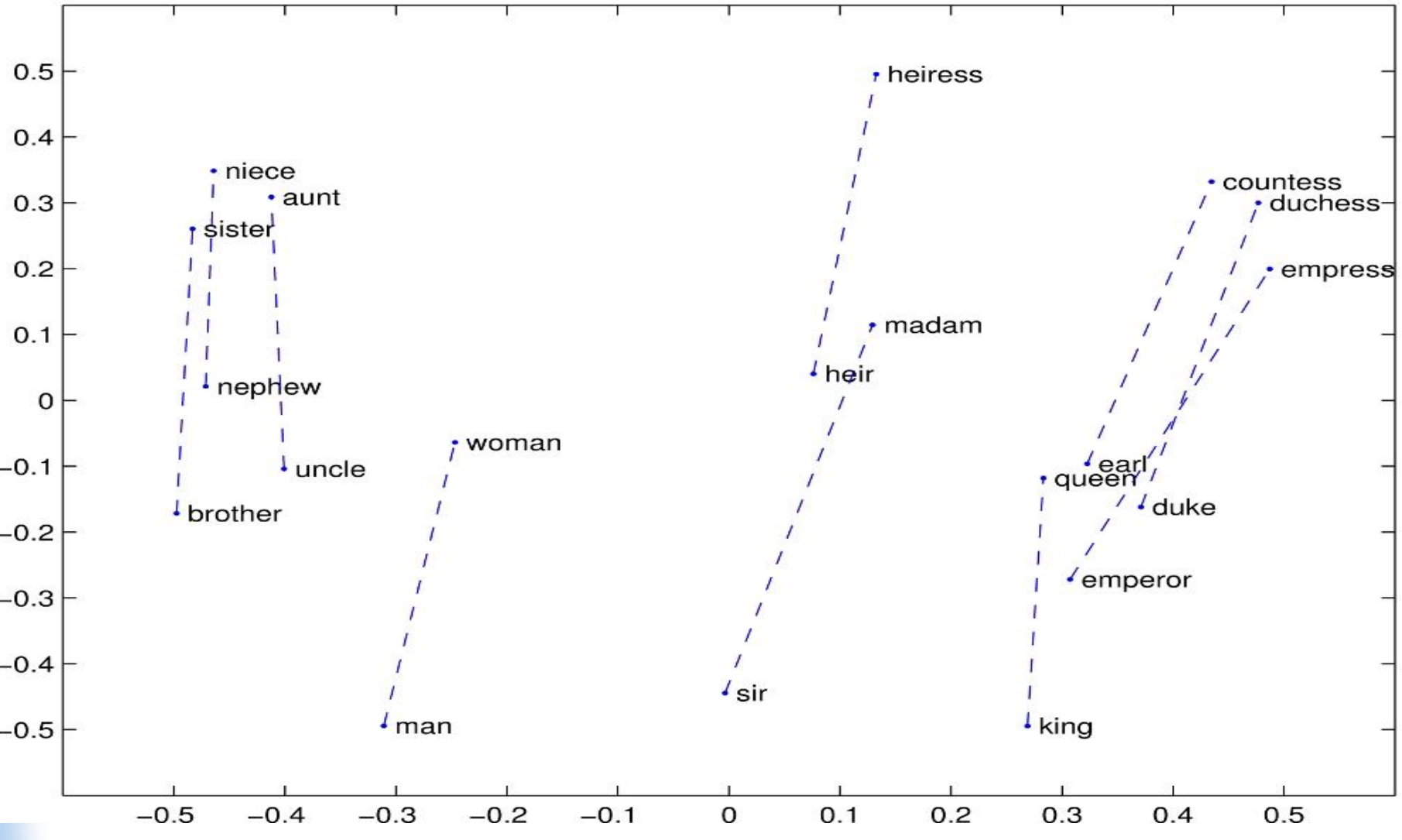
$\overrightarrow{\texttt{king}} - \overrightarrow{\texttt{man}} + \overrightarrow{\texttt{woman}}$ is close to $\overrightarrow{\texttt{queen}}$

$\overrightarrow{\texttt{Paris}} - \overrightarrow{\texttt{France}} + \overrightarrow{\texttt{Italy}}$ is close to $\overrightarrow{\texttt{Rome}}$

- For a problem a:a*::b:b*, the parallelogram method is:

$$\hat{b}^* = \underset{x}{\text{argmax}}\ \text{distance}(x, a^* - a + b)$$

Structure in GloVE Embedding space



king − man + woman is close to queen

Caveats with the parallelogram method

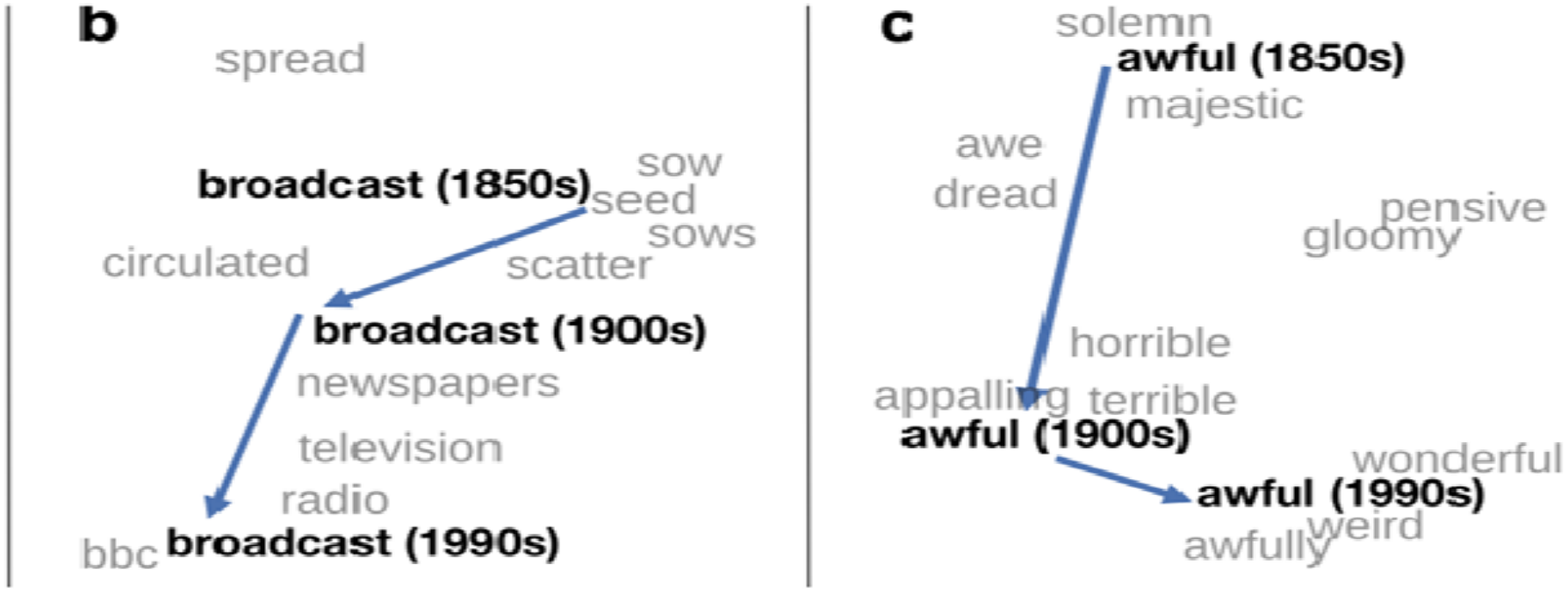
- It only seems to work for frequent words, small distances and certain relations (relating countries to capitals, or Parts-of-Speech), but not others. (Linzen 2016; Gladkova et al. 2016; Ethayarajh et al. 2019a)

- Understanding analogy is an open area of research (Peterson et al. 2020)

Embeddings as a window onto historical semantics

Train embeddings on different decades of historical text to see meanings shift.

~30 million books, 1850-1990, Google Books data



William L. Hamilton, Jure Leskovec, and Dan Jurafsky. 2016. Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change. Proceedings of ACL.

Embeddings reflect cultural bias!

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." In *NeurIPS*, pp. 4349-4357. 2016.

- Ask “Paris : France :: Tokyo : x”
  - x = Japan
- Ask “father : doctor :: mother : x”
  - x = nurse
- Ask “man : computer programmer :: woman : x”
  - x = homemaker
- Algorithms that use embeddings as part of e.g., hiring searches for programmers, might lead to bias in hiring

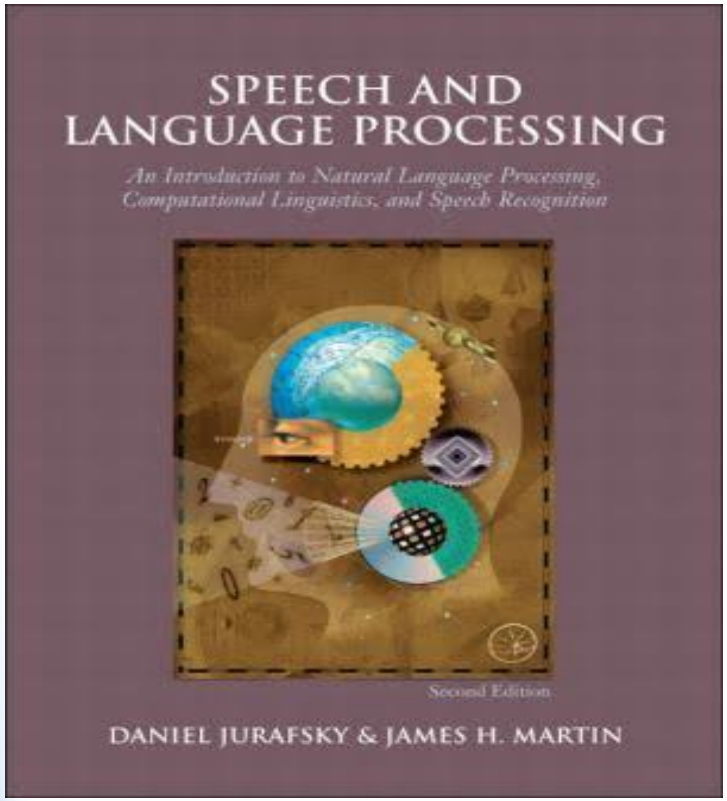
Historical embedding as a tool to study cultural biases

Garg, N., Schiebinger, L., Jurafsky, D., and Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. Proceedings of the National Academy of Sciences 115(16), E3635–E3644.

- Compute a **gender or ethnic bias** for each adjective: e.g., how much closer the adjective is to "woman" synonyms than "man" synonyms, or names of particular ethnicities.
  - Embeddings for **competence** adjective (*smart, wise, brilliant, resourceful, thoughtful, logical)* are biased toward men, a bias slowly decreasing 1960-1990
  - Embeddings for **dehumanizing** adjectives (barbaric, monstrous, bizarre) were biased toward Asians in the 1930s, bias decreasing over the 20th century.
- These match the results of old surveys done in the 1930s

Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.